웹서비스설계 보고서 작성 양식

JWT 게시판 구현 보고서

01 서비스 소개와 구현 범위

학번: 202117321 이름: 안태형 제출일: 2026-10-21

서비스 이름 STRENGTH & DIET

기술 조합과 버전: Vue 3 (Node.js 24.21.0) / Spring Boot (Java 21.0.12.1) / SQLite

GitHub 저장소 URL: https://github.com/202117321antaehyoung/Assignment-1-Implementing-CRUD-and-Login

소스 코드 기준: main 브랜치 최신 커밋 (최종 커밋 SHA 는 LMS 에 별도 제출)

본 서비스는 데드리프트,스쿼트,벤치프레스 3 대 운동의 리프팅 중량과 매일 식단을 기록할 수 있는 일지 서비스입니다. 작성자의 훈련 기록을 다른 접속자로부터 보호하기 위해 JWT 기반의 인증 및 인가 시스템을 적용했습니다. 사용자 인터페이스의 반응성을 높이기 위해 프론트엔드로 Vue 를, 안정적이고 보안이 강력한 API 제공을 위해 백엔드로 Spring Boot 를 사용했습니다. App.vue 를 수정하여 스트렝스 테마에 맞는 배경 이미지와 CSS 투명도 효과를 추가로 적용했습니다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	[완료]	[api.js, App.vue]	[비로그인 시에도 목록과 내용 정상 조회됨]
로그인과 JWT 발급	[완료]	[AuthService.java, api.js]	[HS256 알고리즘 적용 완료]
인증과 내 정보 조회	[완료]	[AuthService.java, api.js]	[Bearer 토큰 파싱 및 검증 성공]
본인 글 작성·수정·삭제	[완료]	[AuthService.java, App.vue]	[프론트 제어 및 백엔드 403 방어 확인]
만료·로그아웃·오류 표시	[완료]	[AuthService.java, api.js]	[401 수신 시 즉시 로그아웃 처리됨]
DB 데이터 유지	[완료]	[data/board.sqlite]	[서버 재시작 및 새로고침 후에도 글 유지됨]

구조와 변경 내역

[브라우저(Vue) -> API 서버(Spring Boot) -> SQLite DB 구조

1. 로그인 흐름: 사용자가 브라우저에서 계정 전송 -> API 서버가 확인 후 JWT 발급 -> 브라우저가 자바스크립트 변수(session.token) 메모리에 토큰 저장.

2. 게시글 수정 흐름: 브라우저가 HTTP 헤더에 Authorization: Bearer <JWT>를 담아 요청 -> 서버가 토큰의 서명, 만료, 대상을 검증(verify)하여 작성자 ID 추출 -> assertOwner 함수에서 글 작성자와 요청자 ID 일치 여부 확인 (다르면 403) -> DB 수정.

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
AuthService.java/ issueToken	501 NOT_IMPLEMENTED 에러를 고정적으로 반환함.	Jwts.builder()를 사용해 필수 클레임(sub, iss, aud, exp)을 포함하고, HexFormat 으로 비밀키를 변환해 HS256 으로 서명한 JWT 를 발급함.	로그인 성공 시 유효한 토큰을 발급하기 위함. **(AI 초안 후 본인 검증 및 수정)**
AuthService.java / currentUserId	무조건 데모 사용자 ID(1L)를 반환함.	Bearer 토큰을 추출하고 Jwts.parser().verifyWith() 및 .requireAudience() 로 엄격히 검증하여 예외(401) 분기 및 진짜 사용자 ID 를 반환함.	요청자의 신원을 확인하고 변조/만료된 토큰을 방어하기 위함. **(AI 초안 후 본인 검증 및 수정)**
AuthService.java/ assertOwner	검사 로직이 없는 빈 함수로 제공됨.	userId 와 authorId 를 비교하여 불일치 시 403 FORBIDDEN 에러를 발생시킴.	타인의 게시글에 대한 비정상적인 수정/삭제 API 호출을 서버 단에서 원천 차단하기 위함. **(AI 초안 후 본인 검증)**
api.js/ login, request	토큰 저장 로직 및 요청 시 헤더 첨부 로직이 없음.	로그인 성공 시 응답 토큰을 메모리(session)에 저장하고, request 시 Authorization 헤더를 조립해 전송함. 401 응답 시 세션을 초기화함.	프론트엔드의 로그인 상태를 관리하고, 새로고침 시 토큰이 해제되도록 보안 요구사항을 충족하기 위함. **(AI 초안 후 본인 검증)**

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
[App.vue/ <template>, <style>]	템플릿의 기본 UI 및 안내 문구 출력.	제목을 '스트렝스 훈련 일지'로 변경하고, 하단 CSS 에 바벨/식단 배경 이미지와 반투명 블러 효과(backdrop-filter)를 추가함.	나만의 게시판 주제(운동 일지)에 맞게 웹서비스의 UI/UX 디자인을 고도화하기 위함. **(AI 초안 후 본인 검증 및 수정)**

로컬 실행 기록

운영체제와 버전: Windows 11

새 폴더 설치 및 환경설정: 새 폴더에 Clone 후, backend-spring/src/main/resources/application.properties 파일에 jwt.secret (64 자리 16 진수) 값 주입 완료.

프론트 빌드: frontend-vue 폴더에서 npm ci 후 npm run build 완료.

백엔드 실행: backend-spring 폴더에서 .₩mvnw.cmd spring-boot:run 정상 실행 완료.

SQLite 파일 경로: backend-spring/data/board.sqlite 자동 생성 및 연동 확인.

확인 결과: 프론트엔드(http://127.0.0.1:4173) 접속 시 백엔드(8000 번 포트) API 연동 정상 작동 확인. 서버 및 브라우저를 재시작해도 SQLite 에 저장된 테스트 게시글 데이터가 정상적으로 유지됨. **(증거 위치: 04 테스트 결과 캡처 참조)**

핵심 코드 설명

3 1 로그인과 JWT 발급

파일 / 함수: AuthService.java / issueToken

코드 발췌

```
public String issueToken(Users.PublicUser user) {
    Date now = new Date();
    byte[] keyBytes = HexFormat.of().parseHex(secret);

    return Jwts.builder()
            .setSubject(String.valueOf(user.id()))
            .setIssuer("webservice-class")
            .setAudience("board-api")
            .setExpiration(new Date(now.getTime() + expirationSeconds * 1000))
            .signWith(Keys.hmacShaKeyFor(keyBytes), SignatureAlgorithm.HS256)
            .compact();
}
```

동작과 응답 코드 설명

입력: 아이디/비밀번호 검증을 통과한 PublicUser 사용자 객체.

처리: Java 17 이상의 HexFormat 을 사용해 64 자리 16 진수 형태의 서버 비밀 키를 32 바이트 배열로 파싱합니다. 이후 Jwts.builder()를 통해 필수 클레임(sub, iss, aud, exp)을 삽입하고, 파싱한 비밀 키를 이용해 HS256 알고리즘으로 서명한 뒤 압축하여 생성합니다.

출력 및 예외: 성공 시 생성된 JWT 문자열을 반환하며, 컨트롤러 단에서 HTTP 상태 200 OK 와 함께 발급된 토큰을 JSON 형태로 응답합니다. 만약 입력된 계정 정보가 틀렸을 경우 토큰 발급 전 컨트롤러에서 401 INVALID_CREDENTIALS 에러를 반환합니다.

3 2 브라우저 토큰 저장과 전송

파일 / 함수: api.js / login, authorizationHeaders, request

코드 발췌

```
// login 함수 내부: 응답 토큰 저장
session.token = response.data.token;
session.user = response.data.user;

// authorizationHeaders 함수: 헤더 조립
export function authorizationHeaders() {
  return session.token ? { Authorization: `Bearer ${session.token}` } : {};
}

// request 함수 내부: API 요청 및 예외 처리
if (response.status === 401) {
  clearSession();
}
```

동작과 응답 코드 설명

입력: 로그인 API 호출 성공 시 백엔드로부터 응답받은 JSON (토큰 및 유저 정보).

처리: XSS 공격 및 상태 초기화를 고려해 로컬 스토리지가 아닌 브라우저 메모리(session 변수)에 토큰을 저장합니다. 이후 API 를 호출할 때마다 authorizationHeaders() 함수가 작동하여 Authorization: Bearer 토큰 형태의 헤더를 전송합니다.

출력 및 예외: 정상 시 토큰이 포함된 HTTP 요청 객체가 만들어집니다. 만약 서버로부터 토큰 만료 또는 인증 실패로 인해 401 상태 코드를 반환받으면, 즉시 clearSession()을 호출해 브라우저의 로그인 상태를 해제 즉 로그아웃 합니다.

서버 인증과 작성자 권한

3 3 서버 JWT 검증

파일 / 함수: AuthService.java / currentUserId

코드 발췌

```
String authHeader = request.getHeader("Authorization");
if (authHeader == null || !authHeader.startsWith("Bearer ")) {
    throw new ApiError(401, "TOKEN_REQUIRED", "인증 토큰이 필요합니다.");
}
String token = authHeader.substring(7);
try {
    byte[] keyBytes = HexFormat.of().parseHex(secret);
    Claims claims = Jwts.parser()
            .verifyWith(Keys.hmacShaKeyFor(keyBytes))
            .requireIssuer("webservice-class")
            .requireAudience("board-api")
            .build().parseSignedClaims(token)
            .getPayload();
    return Long.parseLong(claims.getSubject());
} catch (ExpiredJwtException e) {
    throw new ApiError(401, "TOKEN_EXPIRED", "토큰이 만료되었습니다.");
} catch (Exception e) {
    throw new ApiError(401, "INVALID_TOKEN", "유효하지 않은 토큰입니다.");
}
```

동작과 응답 코드 설명

입력: 클라이언트가 API 를 요청할 때 전달한 HttpServletRequest 의 헤더 정보.

처리: Authorization 헤더에서 Bearer 문자열을 제거해 순수 토큰을 추출합니다. Jwts.parser().verifyWith()를 통해 서버의 비밀 키로 서명을 검증하며, .requireIssuer() 및 .requireAudience()를 통해 발급자와 대상이 일치하는지 확인합니다.

출력 및 예외: 검증 성공 시 토큰의 sub 클레임에서 사용자 ID 를 Long 타입으로 반환 합니다. 토큰 누락 시 401 TOKEN_REQUIRED, 만료 시 401 TOKEN_EXPIRED, 서명 위변조 시 401 INVALID_TOKEN 을 예외로 반환합니다.

3 4 작성자 권한 검사

파일 / 함수: AuthService.java / assertOwner

코드 발췌

```
public void assertOwner(long userId, long authorId) {
  if (userId != authorId) {
    throw new ApiError(403, "FORBIDDEN", "본인의 게시글만 수정하거나 삭제할 수 있습니다.");
  }
}
```

동작과 응답 코드 설명

입력: 토큰 검증(currentUserId)을 통과한 현재 로그인 유저의 userId 와, 수정/삭제하려 게시글을 작성했던 원본 작성자의 authorId.

처리: 서버 에서 두 ID 값을 비교합니다. 클라이언트의 UI 변조 여부와 무관하게 인가(Authorization)를 수행합니다.

출력 및 예외: 두 ID 가 일치하면 메서드를 통과하여 정상적인 데이터 수정 및 삭제를 진행합니다. 값이 다를 경우 403 FORBIDDEN 에러를 반환해 남의 글 수정 및 삭제를 차단합니다.

테스트 결과

T01 부터 T17 까지 수행하고, 실제 응답과 증거의 위치를 기록하세요.

실행 환경: Windows 11 / Node.js 24.21.0 / Java 21.0.12.1 (JDK 21) / Chrome 브라우저 /
백엔드: .₩mvnw.cmd spring-boot:run 및 프론트엔드: npm run preview

로컬 배포 확인: 새 폴더에 git clone 후 backend-spring 폴더로 이동하여 .\mvnw.cmd spring-boot:run 명령으로 백엔드(8000 번)를 실행함. 새 터미널을 열어 frontend-vue 폴더로 이동 후 npm ci 로 의존성 설치, npm run build 로 정적 빌드, npm run preview 명령을 실행 완료함. 브라우저로 http://127.0.0.1:4173에 접속하여 로그인 및 게시글 조작 등 백엔드 API 와의 연동 및 정상 동작을 최종 확인 완료함. (증거: T18 빌드 성공 로그 및 4173 API 연결 화면 캡처 이미지 첨부)

테스트 일시 및 코드 버전: 2026 년 10 월 20 일 / main 브랜치 최신 커밋

토큰 TTL: 일반 테스트 값 90 초 / 만료 테스트 값 3 초 (T10 만료 테스트 진행 시, 환경변수 변경 대신 AuthService.java 의 issueToken 메서드 내부 만료 시간 계산 로직을 now.getTime() + 3000 으로 코딩 적용하여 백엔드 재시작 후 3 초 만료 테스트 진행, 완료 후 다시 원래 로직으로 원상 복구함)

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	비로그인 상태로 목록 및 상세 조회	200, 정상 공개 조회	200 / 해당 없음	PASS	T1.png
T02	Alice 계정(alice@class.test)으로 정상 로그인	200, JWT 및 사용자 정보 반환	200 / 해당 없음	PASS	T2.png
T03	틀린 비밀번호 및 없는 이메일로 로그인 시도	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	PASS	T3.png
T04	토큰 없이 /me 및 글 등록 요청 전송	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	PASS	T4.png
T05	Alice 로그인 후 새 글 작성 및 /me 조회	201 Created (Location 헤더 포함), /me 정상	201 / 해당 없음 200 / 해당 없음	PASS	T5.png
T06	Alice 토큰으로 본인이 쓴 글 수정 및 삭제	200(수정) 및 204(삭제)	200 / 해당 없음 204 / 해당 없음	PASS	T6-1.png, T6-2.png
T07	Bob 으로 로그인 후 Alice 의 글 수정 시도	403 FORBIDDEN, 원문 유지	403 / FORBIDDEN	PASS	T7.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T08	Bob 으로 로그인 후 Alice 의 글 삭제 시도	403 FORBIDDEN, 글 유지	403 / FORBIDDEN	PASS	T8.png
T09	Application 탭에서 토큰 서명부 임의 변조 후 요청	401 INVALID_TOKEN, 데이터 변경 없음	401 / INVALID_TOKEN	PASS	T9-1.png, T9-2.png
T10	TOKEN_TTL=3 설정 후 4 초 대기 뒤 변경 요청	401 TOKEN_EXPIRED, 로그인 해제	401 / TOKEN_EXPIRED	PASS	T10-1.png, T10-2.png
T11	로그아웃 버튼 클릭 후 /me 다시 요청	401 TOKEN_REQUIRED (헤더에 토큰 없음)	401 / TOKEN_REQUIRED	PASS	T11-1.png, T11-2.png
T12	Bob 으로 글 등록 시 JSON 본문에 authorId:1 추가	400 BAD_INPUT, 위조 거부	400 / BAD_INPUT	PASS	T12-1.png, T12-2.png
T13	본인 토큰으로 제목 빈칸, 내용 공백으로 전송	400 BAD_INPUT, DB 유지	400 / BAD_INPUT	PASS	T13-1.png, T13-2.png
T14	글 번호 9999(없는글) 조회/수정 및 abc(형식오류) 조회	404 NOT_FOUND 및 400 BAD_INPUT	404 / NOT_FOUND 400 / BAD_INPUT	PASS	T14-1.png, T14-2.png, T14-3.png, T14-4.png
T15	글 생성 후 Spring 백엔드 완전 종료 후 재실행	200, 기존 글과 작성자 유지 확인	200 / 해당 없음	PASS	T15-1.png, T15-2.png, T15-3.png
T16	로그인 상태에서 F5(새로고침) 실행	공개 글 유지, 로그인 상태 해제	200 / 해당 없음	PASS	T16-1.png, T16-2.png
T17	무토큰 수정 요청(401) 및 Bob 의 본인 글 정상 CRUD	무토큰 401, 본인작업 200/204	401 / TOKEN_REQUIRED 200 / 204	PASS	T17-1.png, T17-2.png, T17-3.png, T17-4.png

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회·로그인·수정	**200**	200 / 해당 없음	T1.png, T2.png, T6-1.png, T16-1.png, T16-2.png, T17-3.png
새 글 등록	**201 및 Location**	201 / 해당 없음	T5.png
본인 글 삭제	**204 및 빈 본문**	204 / 해당 없음	T6-2.png, T17-4.png
입력 오류·추가 작성자 필드	**400 BAD_INPUT**	400 / BAD_INPUT	T12-1.png, T12-2.png T13-1.png, T13-2.png T14-3.png, T14-4.png
잘못된 계정	**401 INVALID_CREDENTIALS**	401 / INVALID_CREDENTIALS	T3.png
토큰 없음·헤더 형식 오류	**401 TOKEN_REQUIRED**	401 / TOKEN_REQUIRED	T4.png, T11-1.png, T11-2.png, T17-1.png, T17-2.png
변조·클레임 오류	**401 INVALID_TOKEN**	401 / INVALID_TOKEN	T9-1.png, T9-2.png
정상 서명의 만료 토큰	**401 TOKEN_EXPIRED**	401 / TOKEN_EXPIRED	T10-1.png, T10-2.png
다른 작성자의 변경	**403 FORBIDDEN**	403 / FORBIDDEN	T7.png, T8.png
없는 글	**404 NOT_FOUND**	404 / NOT_FOUND	T14-1.png, T14-2.png

토큰 없이 API 를 요청하거나 만료된 토큰으로 접근했을 때 반환된 401 Unauthorized 응답의 Response Headers 를 확인한 결과, 서버가 인증 방식을 요구하는 WWW-Authenticate: Bearer 헤더를 포함하여 반환하고 있음을 확인했습니다. (T9-1.png, T11-1.png)

로그인이 강제로 해제되는지 그렇지 않은지가 가장 큰 차이 입니다. 401 (인증 실패/만료/없음) 응답을 받으면 프론트엔드 메모리에 보관 중이던 토큰과 사용자 정보가 삭제되며, 로그아웃 됩니다. 반면, 403 (권한 부족) 응답의 경우 로그인 인증 자체는 유효하므로 로그인 상태가 해제되지 않고, 화면 상단에 권한 부족 에러 알림 메시지만 출력됩니다.

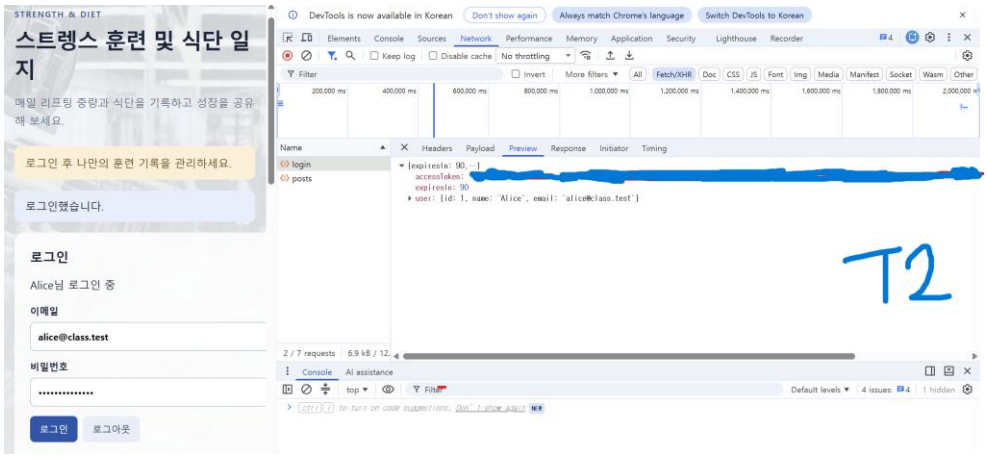
Bob 이 Alice 의 글에 대해 수정 및 삭제를 시도하여 403 FORBIDDEN 응답을 받았습니다. 실패 응답 직후 해당 게시글을 다시 조회한 결과, 기존의 글 내용과 작성자인 Alice 가 원문 그대로 유지되었습니다.

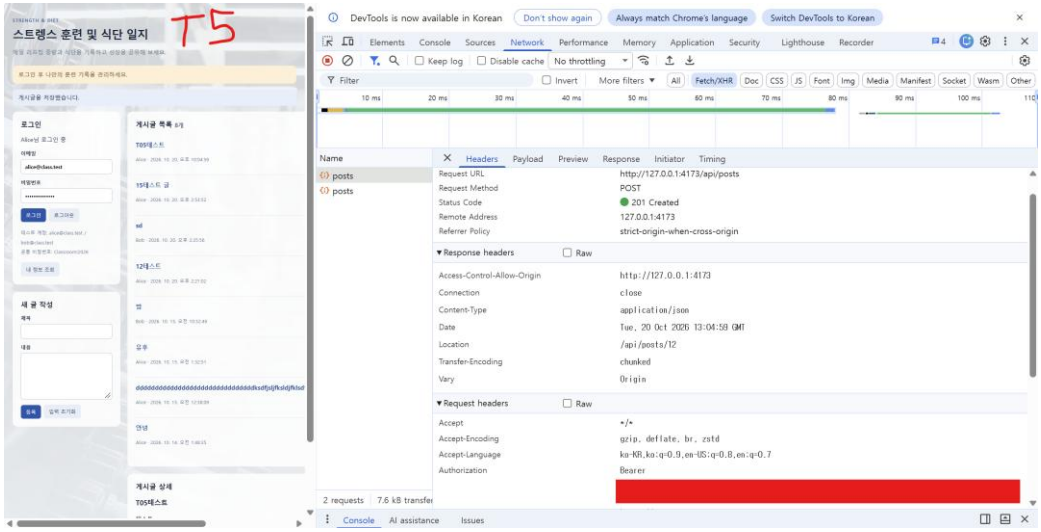
비로그인 상태로 글 삭제 요청을 보냈을 때는 인증 필터에서 차단되어 401 / TOKEN_REQUIRED 가 반환되었습니다. 반면, 로그인 상태로 존재하지 않는 글 번호(9999 번) 수정 시, 사용자 인증은 통과하였으나 DB 게시글 조회 로직에서 대상을 찾지 못해 404 / NOT_FOUND 가 반환되었습니다.

테스트 증거

로그인과 토큰 전송

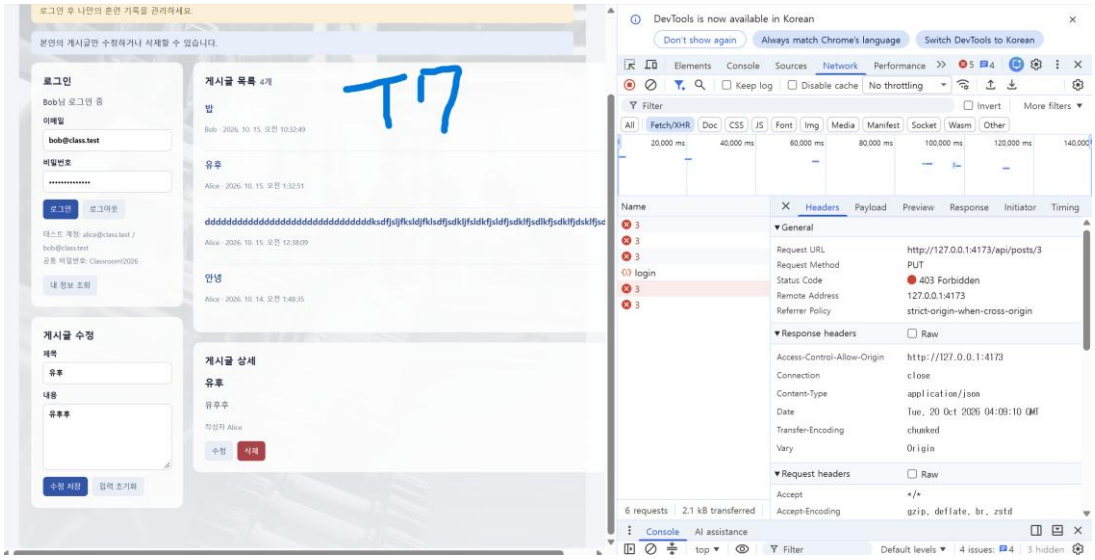
Alice 계정으로 로그인 시 200 OK 상태 코드와 함께 서버로부터 accessToken 과 사용자 정보가 담긴 JSON 수신을 확인했습니다. 또한, 로그인 후 새 글을 작성(POST /api/posts)하거나 수정(PUT)할 때, Request Headers 의 Authorization 필드에 Bearer {토큰} 형태로 토큰 첨부 후 전송을 확인했습니다.

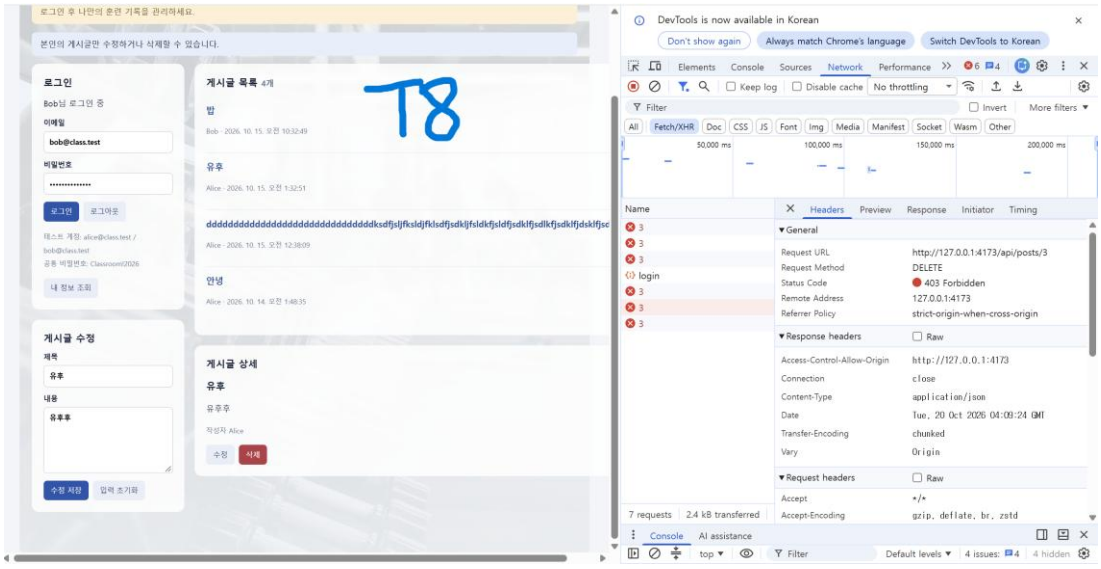




다른 작성자의 수정과 삭제

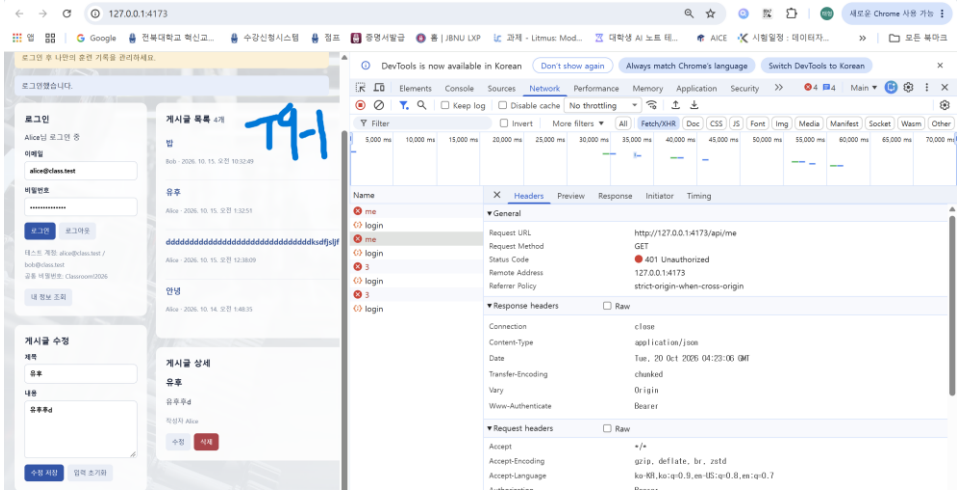
Bob 계정으로 로그인한 상태에서 Alice 가 작성한 게시글에 대해 수정을 시도했을 때, 서버 권한 검증 로직에 의해 403 FORBIDDEN 이 반환되었습니다. 삭제를 시도했을 때도 403 FORBIDDEN 응답과 함께 요청이 거부되어 다른 작성자의 데이터가 안전하게 보호됨을 확인했습니다.

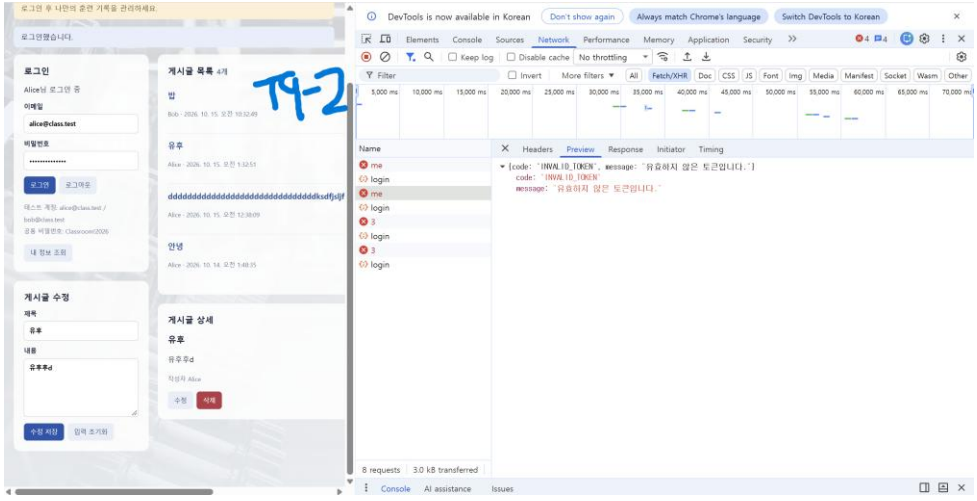


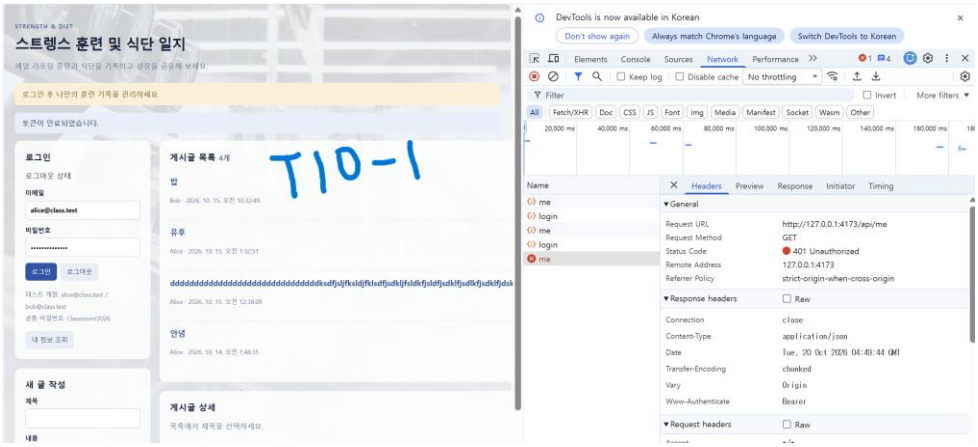


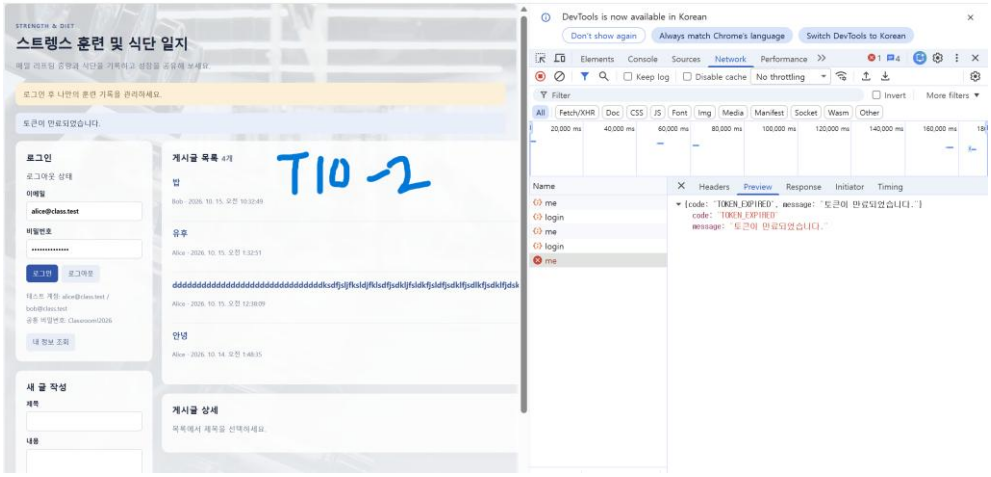
토큰 변조와 만료

브라우저 Application 탭에서 토큰의 서명부 마지막 글자를 변조한 뒤 보호 API(/api/me)를 요청한 결과, 서버가 위조를 감지하고 401 INVALID_TOKEN 을 반환했습니다. 또한 백엔드 setExpiration 부분에 now.getTime() + 3000 을 적용하여 3 초 만료 토큰을 발급받은 뒤 3 초 초과 후 요청을 보냈을 때, 401 TOKEN_EXPIRED 응답이 반환되며 프론트엔드 화면의 로그인 상태가 강제 해제됨을 확인했습니다.


T9-1

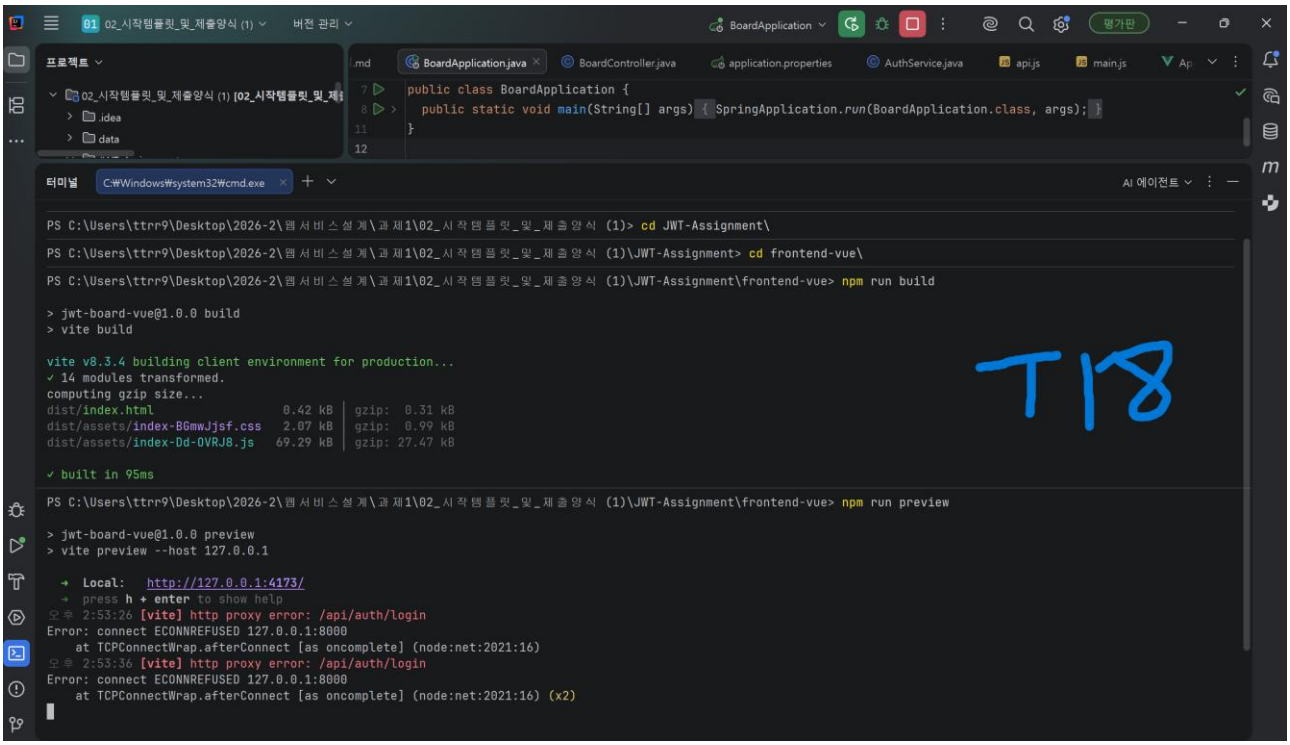

T9-2

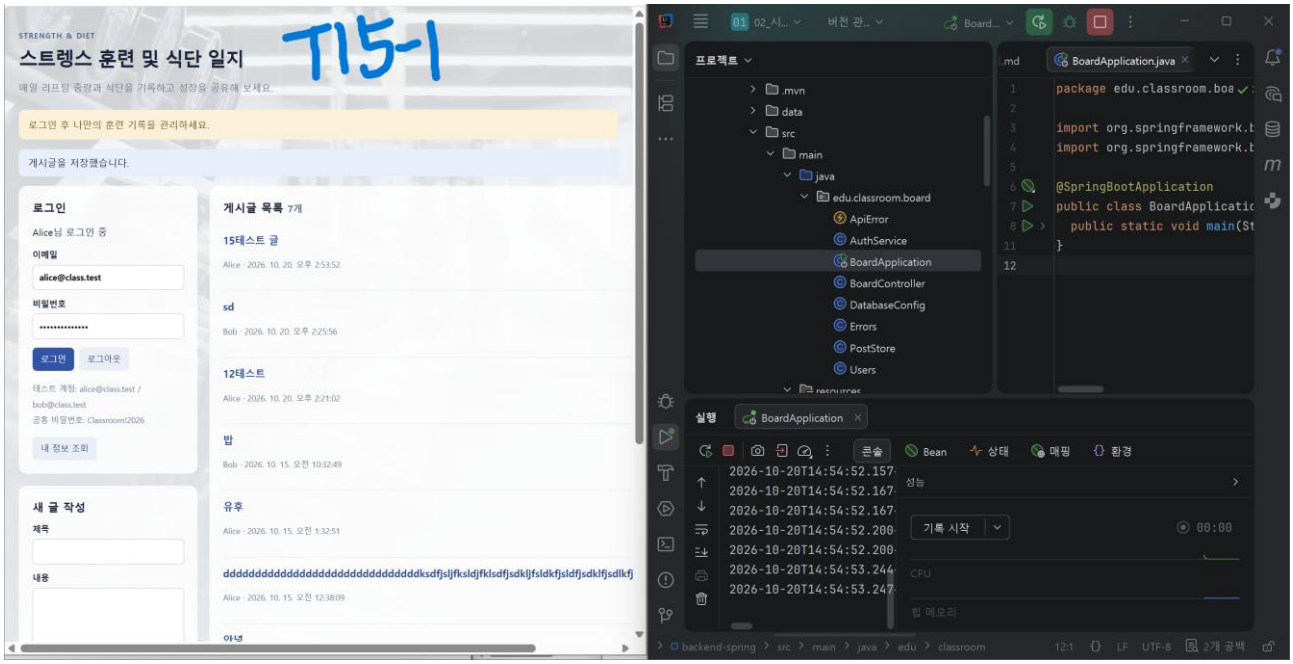

T10-1

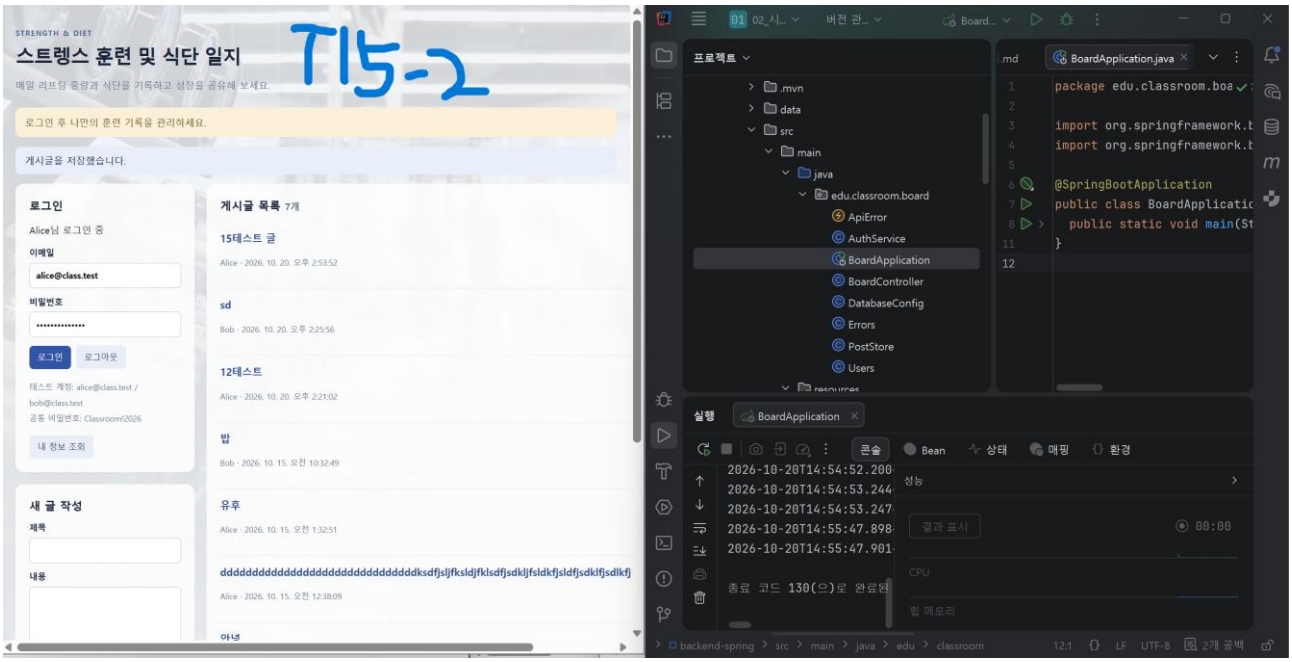

T10-2

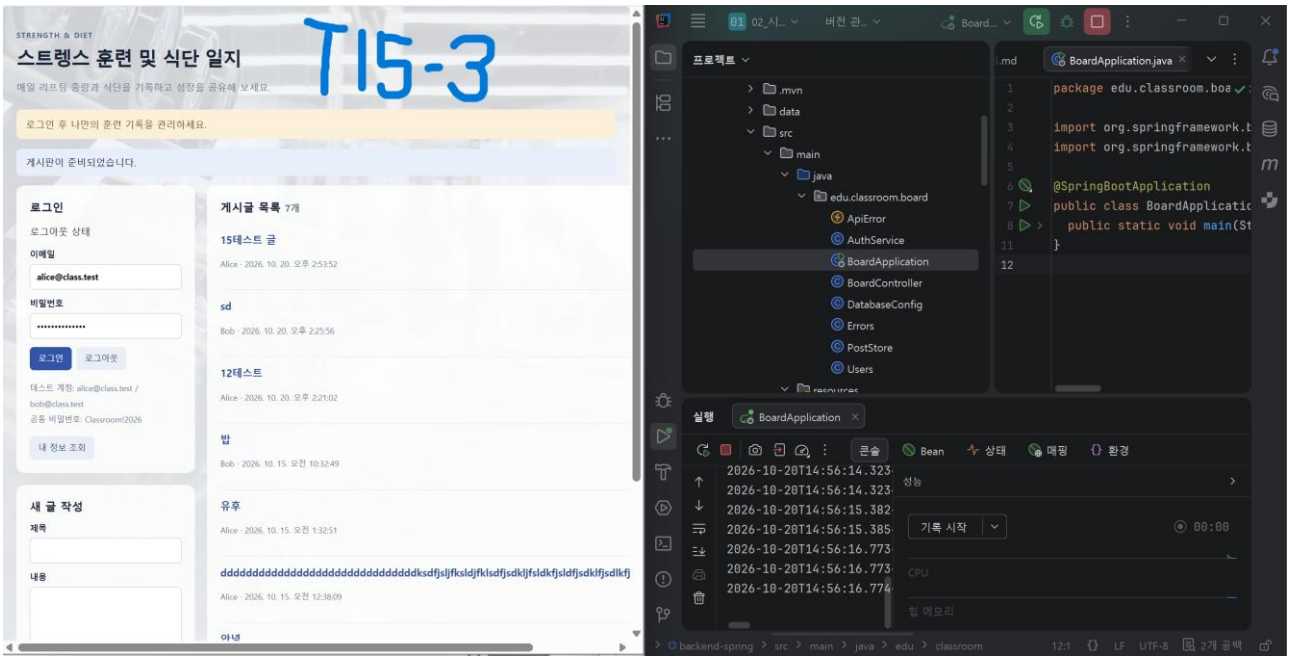
자동 검사와 로컬 배포 증거

자동 검사 도구 대신 직접 npm run build 와 npm run preview 명령어를 실행하여 프론트엔드를 정적 빌드 및 배포하였습니다. 별도의 개발 서버 없이 로컬 배포 환경(4173 번 포트)에서 백엔드(8000 번 포트) API 가 정상 연동됨을 확인했습니다. 백엔드 서버 프로세스를 완전히 종료한 후 다시 실행하였을 때, 기존에 작성했던 게시글 데이터와 작성자 정보가 저장되어 화면에 유지됨을 증명했습니다.









AI 활용과 참고 자료

AI 사용 여부: [사용]

[전체 사용 범위 요약]

본 과제를 수행하면서 과제에 대한 이해, 그리고 코드 작성, 보고서 작성 모두 ai 를 적극 활용했습니다. 먼저 과제 안내 pdf 와 기본 코드를 제미나이 에게 보내 인식시키고 이 과제가 요구하는 내용과 내가 과제를 수행하면서 알아야 할 것을 정리하도록 했습니다. 그리고 과제를 위해 추가하거나 수정해야 하는 코드를 ai 가 작성하게 하고 작성된 코드를 이해한 후 과제 요구사항에 충족하면 그대로 사용하였고, 테스트 시 오류가 발생하면 오류증상을 보내고 왜 그런지 다시 이해한 후 직접 수정하거나 ai 가 수정해서 보내준 코드를 적용했습니다. 예로는 JWT 인증 만료 시간 설정의 논리적 오류(버그)를 해결하기 위한 디버깅, 프론트엔드 보안 변수 은닉으로 인해 막힌 위조 테스트(T12)를 우회하는 해킹 스크립트 작성, 그리고 수많은

테스트 결과와 HTTP 응답 코드를 보고서 양식에 맞게 표로 구조화하는 과정 등등 이 있습니다. AI 가 제시한 코드나 해결책을 무비판적으로 복사하지 않고, 프론트엔드-백엔드 간의 통신 흐름과 보안 개념을 제가 먼저 이해한 뒤 로컬 테스트 환경에서 직접 검증하고 적용 또는 테스트 후 원상복구하는 방식으로 역할을 분담했습니다. 보고서 작성시에도 보고서 양식과 작성한 코드내용을 참고하여 작성시켰고 작성된 내용에 오류나 어색한 문장을 수정하면서 적용 및 진행하였습니다.

도구와 모델	사용 목적	반영한 파일 또는 보고서 항목	반영 방식
Gemini	디버깅 및 테스트	JWT-Assignment₩backend-spring₩src₩main₩java₩edu₩classroom₩board₩AuthService.java	참고 및 그대로 반영 (일부 수정)
Gemini	코드 생성 및 테스트	JWT-Assignment₩frontend-vue₩src₩api.js, JWT-Assignment₩frontend-vue₩src₩App.vue	참고 및 그대로 반영 (일부 수정)
Gemini	문장 정리 및 문서화	보고서 전체 (테스트 결과 표 및 서술형 답변)	참고 및 일부 수정 후 반영

대표 사례

1.

도구 / 날짜: Gemini / 2026 년 10 월 20 일

요청한 내용: T10 토큰 만료 테스트를 위해 코드를 수정한 뒤 4 초 이상 대기해도 만료 에러가 뜨지 않고 정상 조회되는 문제에 대해 디버깅을 요청함.

AI 가 제안한 내용: 기존 코드(expirationSeconds * 1000)에서 제가 1000 을 3000 으로 바꾼 것이 원인임을 지적함. 기존의 1000 은 밀리초(ms) 단위 변환을 위한 값인데, 이를 곱해버리면서 토큰 수명이 270 초(90 초 * 3000)로 뻥튀기되었다고 설명하며, now.getTime() + 3000 으로 수정할 것을 제안함.

실제 반영 위치: JWT-Assignment\backend-spring\src\main\java\edu\classroom\board\AuthService.java/ issueToken

내가 확인하고 수정한 내용: 타임스탬프 로직에서 expirationSeconds 가 곱해져 있어서 토큰 수명 계산 로직에 의해 토큰 수명이 길어졌다는 것을 알게 되었습니다. AI 가 짜준 코드(now.getTime() + 3000)를 복사해 테스트를 진행하였고, 만료 테스트 증거 캡쳐 후 기존의 정상적인 동적 만료 시간 코드(expirationSeconds * 1000)로 원상 복구하였습니다.

검증 방법과 결과: T10 / GET /api/me / 401 TOKEN_EXPIRED / 하드코딩 수정본 적용 시 3 초 초과 후 인증 실패 및 자동 로그아웃됨을 확인 후 원상복구 완료

2.

도구 / 날짜: Gemini / 2026 년 10 월 20 일

요청한 내용: T12 시나리오(Bob 이 authorId 를 1 로 위조) 검증을 위해 개발자 도구 콘솔에서 fetch API 를 날렸으나 session is not defined 에러가 발생하여 우회 방법을 요청함.

AI 가 제안한 내용: 프론트엔드 모듈 스코프 은닉화 때문에 콘솔에서 전역 변수로 접근할 수 없음을 설명함. 대안으로 src/api.js 의 게시글 생성(create) 함수 내부에 일시적으로 input.authorId = 1; 이라는 코드를 주입하여 강제로 위조 데이터를 서버로 보내는 방식을 제안함.

실제 반영 위치: JWT-Assignment₩frontend-vue₩src₩api.js,

/ create 함수 내부

내가 확인하고 수정한 내용:

보안상 프론트엔드 변수가 보호되고 있다는 점을 이해하고, 서버의 방어 로직을 찌르기 위해서는 프론트엔드 코드를 직접 오염시켜 요청을 보내야 함을 배웠습니다. 제안된 코드를 삽입하여 백엔드의 위조 방어망을 직접 공격해 보았고, 백엔드 검증 로직이 확실하게 동작함을 눈으로 확인한 뒤, 오염시켰던 input.authorId = 1; 코드를 삭제하여 시스템을 정상화했습니다.

검증 방법과 결과: T12 / POST /api/posts (위조 데이터 포함) / 400 BAD_INPUT / 백엔드가 위조 데이터를 거부함을 확인 후 삽입된 테스트 코드 삭제 완료

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
01_과제공지.pdf	프로젝트 전체 세팅 및 요구사항 분석	백엔드 포트 번호 확인, 파일 DB 실행 조건, 응답 코드 및 시나리오(T01~T17) 작성, 과제 가이드라인 확인

이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

클라이언트가 보내는 JSON 본문의 authorId 를 그대로 신뢰하면, 악의적인 사용자가 API 요청 시 authorId 값을 다른 사람의 ID 로 조작하여 타인 명의로 글을 작성(Spoofing)하는 보안 문제가 발생합니다. 이를 차단하기 위해 프론트엔드에서 넘어온 값을 무시하고, AuthService.java 의 currentUserId(HttpServletRequest request) 메서드를 호출하여 HTTP 헤더에 담긴 JWT 의 Payload(sub)에서 검증된 사용자 ID 를 직접 추출하여 사용합니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

프론트엔드(App.vue)에서 canEdit 함수를 통해 타인의 게시글일 경우 수정/삭제 버튼을 UI 에서 숨기더라도, Postman 이나 브라우저 콘솔 등을 통해 백엔드 API 로 직접 PUT 이나 DELETE 요청을 전송할 수 있기 때문입니다. 따라서 서버 단의 방어가 필수적이며, AuthService.java 의 assertOwner(long userId, long authorId) 함수를 통해 서버에 요청을 보낸 검증된 사용자 ID 와 DB 에 저장된 게시글의 원작성자 ID 를 비교하고, 불일치할 경우 403 FORBIDDEN 를 발생시켜 작업을 차단합니다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

decode 는 단순히 Base64 로 인코딩된 문자열을 사람이 읽을 수 있는 JSON 객체로 푸는 과정으로 위변조 여부를 보장하지 못합니다. 반면 verify 는 서버가 가진 비밀 키(Secret Key)를 사용해 서명(Signature)을 복호화하고, 토큰 내용이 중간에 변조되지 않았음을 수학적으로 증명하는 과정입니다. 내 코드(AuthService.java 의 currentUserId)는 JJWT 라이브러리의 파서를 활용해 verifyWith(Keys.hmacShaKeyFor(keyBytes))로 서명의 유효성을 검증할 뿐만 아니라, requireIssuer("webservice-class")와 requireAudience("board-api") 메서드를 통해 토큰의 발급자(iss)와 사용 대상(aud) 클레임까지 완벽하게 일치하는지 엄격한 다중 검증을 수행합니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃: 프론트엔드(api.js)의 clearSession() 함수가 호출되어 메모리 변수(session)에 보관 중이던 토큰과 사용자 정보를 비워냅니다.

토큰 만료: 서버 측 변화로, JWT 내부의 exp 시간이 현재 시간보다 과거가 되어 백엔드 파싱 시 ExpiredJwtException 이 발생하고 401 TOKEN_EXPIRED 를 반환합니다.

새로고침(F5): 브라우저 특성상 자바스크립트 실행 환경이 초기화되면서 메모리 변수(let session)가 날아가 자연스럽게 로그아웃 상태가 됩니다.

이미 복사된 토큰: JWT 는 서버가 상태를 저장하지 않는 Stateless 방식을 따르므로, 내가 프론트엔드에서 로그아웃이나 새로고침을 하더라도 이미 복사해 둔 토큰 원문은 그 토큰 자체의 수명(exp)이 다하기 전까지는 서버에서 계속 유효한 신분증으로 취급되어 API 인증을 통과하게 됩니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

T10 토큰 만료 테스트를 진행할 때, 만료 시간을 3 초로 줄이기 위해 코드를 수정했음에도 시간이 지나도 에러 없이 인증이 성공하는 오류가 발생했습니다. 원인을 분석해 보니, AuthService.java 의 issueToken 함수 내부에 expirationSeconds * 1000 (기본 90 초 * 1000 = 90,000ms)이라는 밀리초 변환 로직이 있는데, 제가 단위 변환의 의미를 간과하고 곱해지는 수인 1000 을 3000 으로 무작정 바꿔버려 토큰 수명이 270 초(90 * 3000)로 뻥튀기된 것이었습니다. 이를 해결하기 위해 타임스탬프 계산 로직의 원리를 정확히 이해한 뒤, 기존 곱셈 로직을 지우고 now.getTime() + 3000 으로 완전히 대체하여 정확히 3 초 뒤에 만료되도록 코딩했습니다. 재검증을 위해 서버 재시작 후 로그인하고 3 초 초과 후 요청을 보냈고, 의도한 대로 401 TOKEN_EXPIRED 응답이 떨어지며 자동 로그아웃되는 것을 확인한 후 원래 코드로 원상 복구하여 문제를 해결했습니다.

현재 남은 문제와 다음에 개선할 점: [없음]

제출 확인: [o] 실제 코드와 설명 일치 [o] 테스트 직접 수행 [o] AI 활용 및 출처 기록 [o] 비밀 키와 토큰 제거

GitHub 확인: [o] 루트 README 작성 [o] 보고서 PDF 와 증거 업로드 [o] README 문서 링크 확인 [o] 교수자 접근 가능 [o] 저장소 URL 과 커밋 SHA 제출